Agent Firewall: Project Plan

Oct 10, 2026 · @Aspirant200715

Summary

Agent Firewall is a local gateway between a coding agent and the user's machine, repos and network, so hidden instructions in content the agent reads cannot reach personal data or trigger actions without permission.

- **Track:** Coding and Agentic Engineering, as a developer tool for coding agents.
- **Nebius and NVIDIA:** the detector and the evaluation victim agent call Nemotron models through Nebius Token Factory. Nano screens quickly and Super reviews borderline cases.
- **Core idea:** the policy engine is the only component that can allow an action, and personal data in any outbound call is blocked or sent for confirmation even when the detector misses the attack.
- **Headline numbers to measure:** attack success rate and personal-data leak rate, with and without the firewall. No results exist yet.

Problem

A coding agent that reads untrusted content, holds private data and can communicate externally can be steered by that content. This combination is often called the lethal trifecta, and the firewall targets its last leg: private data leaving the machine.

Case	What happened	Source
Eight coding agents analysed	Arbitrary command execution in five and data exfiltration in four, all without user approval. SSH keys were stolen through a public Git issue.	Takedown paper
Diagram renderer as exfil channel	An API key left the machine inside a Mermaid diagram that fetched an image from an attacker's server.	Takedown paper

Case	What happened	Source
Antigravity browser agent	Hidden text on a web page led the agent to build a malicious URL. The default allowlist included webhook.site, a service that records every request.	BD Tech Talks
Nx supply-chain attack, August 2025	Malicious npm versions ran installed AI CLIs with permission-skipping flags to scan filesystems for secrets. Wiz reported success in hundreds of cases, though provider guardrails sometimes stopped it.	InfoQ, The Register
Agent Skills	Malicious instructions hidden in skill files and scripts exfiltrate data. Every line of a skill is read as an instruction, so detectors that look for instructions in data cannot cover them.	Agent Skills paper
Personal data in banking agents	Models often refuse to leak passwords but still disclose other personal data.	arXiv study
Guardrail bypass in Agent Builder	A prompt injection bypassed a guardrail step, and CRM data was emailed to the attacker.	PromptArmor

Two lessons shape the design. Data leaves through channels that do not look like network calls, and detection alone fails by design for some inputs.

Solution and differentiation

The firewall puts a content-aware gate in front of every tool call: it asks what is going out and why, not only where it is going.

For each call it checks the task's scope, the arguments, any personal data inside them, and whether the session recently read untrusted content. It then allows, asks the user to confirm, or blocks, and logs the decision.

NVIDIA's OpenShell already provides sandboxed agents with policy-enforced egress. According to NVIDIA's docs, its proxy asks the policy engine about the destination, the port and the calling binary, and the engine returns allow or deny. The firewall adds:

- payload inspection for personal data, using canary strings and patterns
- session taint tracking, so a push or network call after untrusted input is escalated
- an audit log that ties each action to its source item and the rule that fired
- an evaluation harness that reports attack success and leak rates against a no-firewall baseline

OpenShell can serve as the sandbox layer if setup goes well. NemoClaw is labelled alpha, so plain Docker is the fallback. Open item: confirm in the OpenShell docs which of the additions above it already offers.

Permissions, taint tracking and the egress rule

Permissions are explicit rules that name the tool, the arguments, the data and the source, and content never grants itself permissions. The most important rule is the personal-data egress rule.

- **Scope by task.** "Fix the failing test in this repo" gets read and write access to that repo only, no personal folders in the home directory, and outbound network only to an allowlist.
- **Egress rule.** If personal data appears in the arguments of any outbound action (network request, commit, pull request, MCP call), the action is blocked or sent for confirmation. This holds even when the detector misses the injection.
- **Always confirm.** git push, deleting files, network POSTs, and any read of a labeled personal-data path.
- **Taint tracking.** A hook after Read and web-fetch calls marks the session when it ingests untrusted or flagged content. Later checks on pushes, network calls and MCP calls escalate while the session is tainted. A hook sees only the tool call, so this is how the gateway knows what content preceded it.
- **Hidden channels.** The egress check inspects URLs and rendered content such as image or diagram links, not only curl and git push. Known request-capture domains are not allowlisted.
- **Personal data labels.** By path (for example SSH keys and finance folders) and by pattern (key formats, emails, phone numbers), with canary strings planted in the synthetic data.
- **Audit record.** Time, task, source item, rule that fired, the call, and the outcome.

Claude Code's PreToolUse hook accepts allow, deny or ask decisions, so the gateway can sit behind a short adapter:

```
#!/usr/bin/env python3
import json, sys
ev = json.load(sys.stdin)           # tool_name, tool_input, session_id
try:
    d = gateway_check(ev)           # policy engine call
except Exception:
    d = Decision("deny", "gateway unavailable (fail closed)")
print(json.dumps({"hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": d.action,  # allow | ask | deny
    "permissionDecisionReason": d.reason}}))
```

Antigravity CLI reportedly supports similar hooks, according to a community guide. Confirm against Google's official docs before building that adapter.

Detection and red-teaming

Detection is advisory and probabilistic: it informs the policy engine and never allows an action itself.

- **Rules layer.** Catches imperative text aimed at the agent, instructions inside comments or docs, hidden text, and requests to send data to a new address or to disable checks.
- **Nemotron detector.** Nano classifies content with a prompt that returns JSON (score, quoted text, reason). Super reviews borderline cases.
- **Why a prompted classifier.** NVIDIA publishes Nemotron Safety Guard models for content safety, topic control and jailbreak detection on Hugging Face. Availability on Token Factory is not verified, and jailbreak detection is not the same task as spotting an injection in a README.

The attack generator plants injections in synthetic repos and documents. All data is synthetic, with fake keys and fake contacts.

Attack family	Example	Channel
Fake setup step	README tells the agent to upload a key file	README
Code comment	Comment asks the agent to read a personal folder	Source file
Dependency doc	Doc requests an outbound POST	Package docs
Hidden web text	White text on a fetched page	Web page

Attack family	Example	Channel
Split instruction	Instruction spread across several issue comments	Issue thread
Poisoned skill file	Exfiltration step hidden in a SKILL.md and its script	Agent skill
Render-based exfil	Data embedded in an image or diagram URL	Rendered output
Hijacked CLI	Install script runs the agent with permission checks off	Supply chain

The first five families come from the original design. The last three are additions drawn from the documented incidents above.

Evaluation

The harness runs a fixed benign task set and the generated attacks through the gateway, and compares every run with a baseline that has no firewall.

- **Victim agent.** A small coding agent driven by Nemotron Super on Token Factory that follows injected instructions, so the baseline is reproducible. Real agents may refuse an injection on their own and blur the comparison. Claude Code connects through the hook adapter for the live demo.
- **Ground truth for leaks.** Fake secrets carry unique canary strings. A leak means a canary appears in an outbound request, commit or pull request body.
- **Where it is measured.** Tools run in a container whose network traffic passes through a logging proxy, so leaks are counted from what actually left. OpenShell is the sandbox if it works in time, otherwise plain Docker.
- **Per attack, record:** whether it reached the tool layer, whether it was blocked or confirmed, and whether personal data left the machine.
- **Outside check.** If a published prompt-injection benchmark fits, run it too, so the numbers do not depend only on attacks written for this defense.

Metric	What it shows
Attack success rate, with and without firewall	Injections that reach a shell or network call. This is the headline number.

Metric	What it shows
Personal-data leak rate, with and without firewall	Canary data that leaves the machine
False block rate on benign tasks	Cost of the firewall to normal coding work
Confirmation rate for content-triggered actions	Whether the gate fires only when it should

No results exist yet. The final false-block target is set after the first harness run.

Failure behavior

The gateway fails closed: if a component is down, sensitive calls are blocked instead of allowed.

If this fails	What the gateway does
Detector is unavailable	Shell, git push, network and personal-data reads are blocked. Read-only work in the task scope may continue, marked unscreened.
Personal data layer is unavailable	Reads outside the repo are blocked.
Taint store is unavailable	The session is treated as tainted.
Policy engine is unavailable	All calls are blocked.
Audit log cannot write	The call is not run, because an action without a record is not allowed.

The taint store row is an addition to the original design.

Demo script

The demo runs under three minutes and ends on the claim that the egress rule limits damage even when detection fails.

1. The agent fixes a failing test in a synthetic repo with no interruption.
2. A synthetic README hides an instruction to upload a fake key file to an outside server. The firewall highlights and quotes the text, then blocks the upload.

3. The audit log entry shows the source, the rule that fired and the decision.
4. The metrics screen shows attack success and leak rates with and without the firewall.
5. One attack gets past the detector. The egress rule and the confirmation prompt still stop the leak.

Limits and risks

The firewall reduces damage from injected instructions; it does not make an agent safe.

What it does not stop:

- a user who confirms a malicious action after reading the quoted text
- an attack that only asks for allowed actions
- a compromised machine or account
- obfuscated commands (for example base64 or an inline script) that hide data from argument checks. The logging proxy catches what leaves the network, but not every route.

Project risks:

- Scope is large. Protect the egress rule, taint tracking, audit log, proxy measurement and demo first.
- NemoClaw is alpha, so sandbox setup may cost time.
- Attacks written by the team may flatter the defense, hence the outside benchmark check.
- Overlap with OpenShell invites the question of what is new. The answer is payload inspection, taint and measured results.

Open items and next steps

Two questions decide how much of the plan survives, and neither is answered yet.

- [ ] Hackathon deadline: sets the length of the schedule.
- [ ] Solo or team: the original design assumes three roles (security, system design, AI/ML).
- [ ] Confirm in the OpenShell docs which additions in this plan it already offers.
- [ ] Confirm Antigravity hook support against Google's official docs.
- [ ] Check whether a Nemotron safety model is hosted on Token Factory. The prompted Nano classifier works either way.

- [ ] Choose how personal data is labeled: by path, by pattern, or both. The plan assumes both.
- [ ] Set the final false-block target after the first harness run.

First concrete steps: get a Nebius key and one working Nemotron call, then write the threat model and the synthetic repos with canary data.

Sources

Taken from search results, not full-page reads. Verify figures and claims against the originals before presenting.

- Takedown: How It's Done in Modern Coding Agent Exploits
- Antigravity prompt injection vulnerability
- InfoQ: s1ngularity Nx attack
- The Register: Nx packages poisoned in AI-assisted supply chain attack
- Agent Skills Enable a New Class of Realistic and Trivially Simple Prompt Injections
- Simple Prompt Injection Attacks Can Leak Personal Data
- PromptArmor: Data Exfiltration in OpenAI Agent Builder via MCP
- GitInject: prompt injection in AI-powered CI/CD
- Claude Code hooks, PreToolUse decisions
- Antigravity CLI hooks guide
- How OpenShell Works, NVIDIA docs
- NVIDIA NemoClaw overview
- NVIDIA NeMo Guardrails and Nemotron Safety Guard
- Nebius: Nemotron on Token Factory